Система аукционов на EJB

Лабораторная работа 3 | Руководство по запуску

Автор: ХУНГАН Клеман Билел | ИТМО

Сервер: Java 17 и WildFly 32.0.1.Final. Клиенты: Java и Groovy 4.0.24. Сборка: Maven 3.9.9. Основной сценарий запуска рассчитан на Windows и PowerShell.

Назначение

Приложение позволяет создавать аукционы, делать ставки, просматривать историю, завершать и отменять аукционы, получать результат. Оба консольных клиента используют один удалённый EJB-компонент и общие данные.

Структура проекта

auction-api: удалённые интерфейсы, объекты результатов и бизнес-исключение.
auction-server: EJB-компонент, сущности JPA и настройки базы.
client-java: клиент и интерактивное меню Java.
client-groovy: клиент и интерактивное меню Groovy.
scripts: установка, запуск, развёртывание и проверки.

Совместимость

Исходные createAuction, placeBid и getAuction сохранены в AuctionService. AuctionManagement добавляет пять операций. Сервер предоставляет обе удалённые бизнес-вью. Старые команды demo, create, bid и show остаются доступны.

Для получения исходников используйте репозиторий, ссылка на который передаётся вместе с этим PDF. Можно клонировать репозиторий либо скачать его архив ZIP. Перейдите в каталог, где находится pom.xml.

1. Установка на новой машине

Установите JDK 17. Откройте PowerShell в корне проекта. Все команды далее выполняются в PowerShell. Проверьте окружение:

```
java --version
javac --version
$env:JAVA_HOME
```

Обе версии должны быть 17. Если выбран другой JDK, задайте путь к своей установленной версии 17. Следующий путь - пример, его нужно заменить при отличии:

```
$env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-17.0.21.9-hotspot"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
```

Выполните первоначальную установку. Интернет необходим для загрузки Maven, WildFly и зависимостей:

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\setup.ps1
```

Ожидаемый результат: BUILD SUCCESS. Затем открывается add-user. Выберите b) Application User, realm ApplicationRealm, имя auction и собственный пароль. Группы можно оставить пустыми. Подтвердите создание пользователя. Если задан вопрос о соединении серверных процессов, ответьте no. Пароль не входит в репозиторий.

Если среда уже установлена

Не создавайте пользователя повторно. Для пересборки используйте:

```
.\compiler.cmd
```

Каталог .runtime хранит локальные инструменты и настройки пользователя. Он не публикуется в GitHub.

2. Сервер, развёртывание и клиенты

Терминал 1, корень проекта:

```
.\demarrer-serveur.cmd
```

Дождитесь WFLYSRV0025 и сообщения started. Оставьте терминал открытым. Сервер доступен только локально; используются порты 8080 и 9990.

Терминал 2, тот же каталог:

```
.\deployer.cmd
```

Ожидается успешное развёртывание auction-server.war. В консоли сервера появляется Deployed "auction-server.war".

Запустите меню Java в терминале 2, а Groovy - в отдельном терминале 3:

```
.\lancer-java.cmd
```

```
.\lancer-groovy.cmd
```

В каждом клиенте введите созданное имя пользователя, например auction, и свой пароль. Оба клиента могут использовать один технический аккаунт.

Пункты меню

1 - создать аукцион.
2 - список открытых аукционов.
3 - информация по ID.
4 - сделать ставку.
5 - история принятых ставок.
6 - завершить аукцион.
7 - отменить аукцион без ставок.
8 - получить результат.
0 - выйти из клиента.

Для подтверждения завершения или отмены введите ДА кириллицей. Суммы - целые рубли; длительность - секунды, от 1 до 86400. ID назначает сервер. Используйте реальный ID из результата создания.

Важно: перезапуск сервера и повторное развёртывание удаляют данные демонстрационной базы H2. Между действиями сценария не перезапускайте сервер.

3. Проверочный сценарий Java + Groovy

1. В Java выберите 1. Введите название Casque, цену 1000, длительность 3600. Запишите полученный ID. Ожидается создание открытого аукциона без ставок.

2. В Groovy выберите 2. Созданный аукцион должен присутствовать в списке. Выберите 4: тот же ID, имя Anna, сумма 1800. Ожидается принятая ставка, цена 1800 и лидер Anna.

3. В Java выберите 3 и тот же ID. Должны отображаться цена 1800 и Anna. Затем выберите 5: история содержит одну принятую ставку. Это показывает обмен данными между двумя клиентами через сервер.

4. В Java выберите 4: тот же ID, имя Ivan, сумма 1700. Сервер должен отказать: ставка должна превышать текущую цену. Цена, лидер и история не изменяются.

5. В Groovy выберите 7 и тот же ID, подтвердите ДА. Ожидается отказ: аукцион с принятой ставкой отменять нельзя.

6. В Java выберите 6, укажите ID и подтвердите ДА. В Groovy выберите 8 для того же ID. Ожидается ПРОДАНО, победитель Anna, итоговая цена 1800 RUB.

7. В Groovy попробуйте ставку 2000 для завершённого аукциона. Ожидается отказ независимо от размера ставки.

8. В Java создайте второй аукцион без ставок. Отмените его через пункт 7 и ДА. В Groovy через пункт 8 проверьте статус ОТМЕНЁН, отсутствие победителя и итоговую цену 0.

Команды исходного прототипа

```
.\scripts\client.ps1 java demo
.\scripts\client.ps1 groovy demo
```

Каждая команда demo создаёт свой новый аукцион. Для общего лота можно использовать команды ниже. Замените 5 реальным ID, возвращённым командой create.

```
.\scripts\client.ps1 java create Casque 1000 3600
$lotId = 5
.\scripts\client.ps1 groovy bid $lotId Anna 1800
.\scripts\client.ps1 java show $lotId
```

4. Проверки, ограничения и диагностика

При работающем сервере и развёрнутом приложении выполните:

```
.\verifier.cmd
```

Проверки создают тестовые аукционы и проверяют бизнес-правила. Выполняйте их до подготовки демонстрационных данных. Подробные результаты предыдущей проверки находятся в docs/Verification_Lab3.md.

На подготовленной версии ранее прошли 14 исходных и 42 новых серверных проверки в Linux. Проверялись также реальные меню Java/Groovy и исходные клиенты с новым сервером. Это не является подтверждением запуска нового меню на компьютере проверяющего: его нужно выполнить по инструкции.

Ограничения

H2 работает в памяти; данные не сохраняются после перезапуска или развёртывания. Один технический аккаунт, без ролей продавца и покупателя. Имя участника вводится как параметр, а не определяется по учётной записи. Нет оплаты, доставки или веб-интерфейса. Срок проверяется сервером при обращениях; отдельного фонового таймера нет.

Типовые проблемы

Connection refused: проверьте, запущен ли WildFly.
Ошибка аутентификации: нужен Application User в ApplicationRealm.
NameNotFound: проверьте успешное развёртывание WAR.
Несовместимые классы: compiler.cmd, затем deployer.cmd.
Порт занят: завершите предыдущий экземпляр WildFly.
Вместо кириллицы вопросительные знаки: используйте Consolas и UTF-8.

```
chcp 65001
[Console]::OutputEncoding = New-Object System.Text.UTF8Encoding
```

Завершение работы

Выберите 0 в обоих клиентах. В терминале WildFly нажмите Ctrl+C. Если Windows спрашивает о завершении командного файла, подтвердите.

Код для проверки: AuctionService.java, AuctionManagement.java, AuctionServiceBean.java, JavaMenu.java, GroovyMenu.groovy и Lab3Checks.java.